Analysis of Prompt Engineering Techniques and Their Algorithmic Effect on Large Language Model Output

1st Sarwagya Acharya
Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
formalsarwagya@gmail.com

2nd Nitesh Pant
Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
np3234@gmail.com

3rd Brishav Joshi
Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
brishavjoshi2081@gmail.com

***Abstract*—Prompt engineering techniques raise benchmark scores for large language models (LLMs). Most reports, though, do not separate the part of the gain that comes from a technique's algorithmic structure from the part that comes from how the prompt happens to be worded or how the output was decoded. We treat four prompting techniques—direct answering, zero-shot and few-shot chain-of-thought (CoT), and self-consistency—as operators on the conditional output distribution and run them through a controlled factorial study across three locally served models (Llama 3.2 1B, Llama 3.2 3B, and Qwen2.5-Coder-3B) on grade-school arithmetic (GSM8K, 100 items) and program synthesis (HumanEval, all 164 problems). On GSM8K, zero-shot CoT lifts the 1B model from 12 to 50 percent, a gain that matches, at the same accuracy, the 41-point jump obtained by scaling from 1B to 3B under direct answering. Few-shot CoT does the same for the 3B and coder models. Self-consistency, in contrast, adds only a smaller margin in uncorrected comparisons. The algorithmic effect ratio on GSM8K sits between 0.82 and 1.00 as a point estimate, though the bootstrap intervals are wide; put plainly, most of the technique gains survive rewording of the prompt. On HumanEval the same ratio ranges from 0.25 to 0.98 and turns degenerate wherever technique effects vanish. Temperature is largely inert for single-sample accuracy at all three scales—the endpoint movements sit inside the bootstrap interval—but self-consistency needs a nonzero temperature and is reported here at its defined setting of 0.7. We close with an evaluation protocol, covering parse rates, token cost, wall-clock latency, and the algorithmic effect ratio, built to separate algorithmic from lexical contributions at small model scale.**

***Index Terms*—prompt engineering, large language models, chain-of-thought, decoding strategies, evaluation, algorithmic analysis.**

I. Introduction

Large language models (LLMs) now sit at the centre of question answering, code synthesis, summarisation, and tool use. How well an LLM application performs is shaped as much by the prompt surrounding a model call as by the weights inside the model [1], [2]. A growing catalogue of *prompt engineering* techniques reports substantial gains on established benchmarks, often with no change to the underlying model: zero- and few-shot exemplars, chain-of-thought (CoT) [3], self-consistency [4], tree-of-thoughts (ToT) [5], ReAct [6], retrieval-augmented prompts [7], and automatically optimised prompts [8].

These gains get reported, but rarely decomposed. The dominant framing is empirical and qualitative: a technique is described by its template, illustrated with selected outputs, and credited with the benchmark delta. What gets lost is the more basic question of *what the technique actually changes inside the model.* Our view is that the principal techniques are best read as algorithmic operations on the conditional distribution $p_\theta(y \mid P, x)$ that an LLM defines over output tokens: a prompt, on this reading, is a control input that perturbs sampling, restructures attention, and conditions intermediate computational traces. The resulting behavioural change often rivals, and sometimes exceeds, the difference between adjacent model scales.

This paper makes four contributions.

- A taxonomy that organises prompt engineering techniques by the algorithmic object they modify: the prompt template, the decoding policy, or the reasoning trace.
- A formal framework that expresses each technique family as a transformation on $p_\theta(y \mid P, x)$ and on the underlying $\arg\max$ or sampling operator.
- A controlled empirical study that isolates the algorithmic effect of prompting from the lexical effect by holding the prompt template fixed and varying the decoding temperature T, and vice versa, at a scale that runs entirely on a single consumer machine.
- An evaluation protocol that we recommend for future prompt engineering research, designed to make algorithmic and lexical contributions separable.

Section II reviews prior surveys and foundational techniques; Section III gives the taxonomy and notation; Section IV the algorithmic effect of each technique; Section V the experimental design; Section VI the results; Sections VII–IX the discussion, limitations, and conclusion.

II. Background and Related Work

The prompt engineering literature spans prompting patterns, decoding-time methods, and reasoning-trace construction. The review follows the same three axes used in the taxonomy of Section III.

A. Prompting patterns as control inputs

Brown et al. [1] showed that scaling produces *in-context learning*: a few exemplars elicit task behaviour without weight updates. Min et al. [2] then showed that ground-truth labels in exemplars contribute less than the input–output format—evidence that prompts act partly as structural scaffolds rather than as miniature training sets.

B. Reasoning-trace construction

Wei et al. [3] introduced chain-of-thought prompting by appending intermediate reasoning steps to exemplars, reporting large gains on arithmetic, commonsense, and symbolic tasks. The algorithmic move here is to reframe generation as sampling a reasoning chain r first, then an answer conditioned on r. Wang et al. [4] extended this with self-consistency: sample many chains and return the most frequent answer, which works because self-consistency beating greedy CoT is itself evidence that the underlying distribution is multimodal. Chains became trees in Yao et al. [5], and Wei et al. [6] interleaved reasoning with tool calls in ReAct. What ties these together is that each exposes intermediate computation to the sampling operator.

C. Decoding-time and retrieval augmentation

Reasoning traces are not the only handle. Temperature, top-k, and nucleus (top-p) sampling [9] parametrise the output distribution directly, and retrieval-augmented generation (RAG) conditions generation on documents fetched at inference time. In our framing RAG is a prompting technique: it rewrites the conditioning context P of $p_\theta(y \mid P, x)$.

D. Common notation and primer

Throughout, P is a prompt template, x the user input, and y the model output; decoding is parameterised by temperature T, a top-k cutoff, and a nucleus threshold p. Section IV formalises these objects.

E. Synthesis and research gap

The recent survey literature marks a shift from cataloguing tricks toward organising the field. Sahoo et al. [10] systematise prompt engineering by application area. Liu et al. [11], by contrast, organise a taxonomy along profile/instruction, knowledge, reasoning/planning, and reliability dimensions. Mei et al. [12] go further still, elevating context engineering to a formal discipline and flagging the asymmetry between consuming and producing complex context as a research priority. And Du et al. [13] show that temperature in multi-sample inference is usually left at a fixed default or tuned on scarce labelled data, with the optimum varying by model, task, and sampling strategy.

Read together, these surveys leave three gaps. No taxonomy, including the most recent ones, sorts techniques by the algorithmic object they modify: the existing surveys organise by application [10], [14] or intended function [11], [15], which places few-shot exemplars and retrieval injection—two operators with the same algorithmic structure, both rewriting the conditioning context—into different cells. Empirical evaluations confound the two factors this paper treats as separable: the prompt is varied while decoding stays at a fixed default, so a reported gain cannot be attributed to the template rather than the decoding configuration, and the interaction between sampling strategy and temperature that [13] documents is rarely controlled for. And no evaluation protocol separates lexical from algorithmic contributions: there is no standard way to report how much of a gain survives rewording, and no requirement to report distribution-level evidence such as per-token divergence, entropy, or realised support width.

The rest of the paper is arranged so that each gap is closed by a specific design choice. The taxonomy of Section III makes the algorithmic object the organising axis, so the functional dimensions of [11] and the application areas of [10] become derived categories rather than primary ones. The controlled ablation of Section V-B is the comparison the surveys motivate but do not run: a fully crossed factorial that varies technique configuration, wording, and decoding independently, turning the confound above into a measurable interaction term. The evaluation protocol of Sections IV-F and V-E reports, alongside accuracy, the parse rates, token cost, wall-clock latency, and the algorithmic effect ratio—the decoding-aware reporting that [13] shows to be necessary and that makes lexical and algorithmic contributions separable even where distribution-level measurements are unavailable.

III. Taxonomy of Prompt Engineering Techniques

A technique that improves a benchmark score tells us little by itself: the improvement may originate at any of three points in the generation pipeline. We therefore classify techniques by the object they modify rather than by their surface form.

A. Three operator classes

Definition 1 (Prompting method). *A prompting method is a triple $M = (\Phi, \Psi, \Omega)$, where Φ is a prompt-space operator acting on the conditioning context, Ψ a decoding-space operator acting on the per-step token distribution, and Ω a trace-space operator acting on the intermediate sequences generated before the answer is emitted; any component may be the identity.*

Definition 1 is deliberately coarse. Two techniques described very differently in prose may occupy the same cell, and two presented as variants may not. Table I assigns the principal techniques to cells and records the cost each incurs.

B. Prompt-space operators

Prompt-space operators rewrite the conditioning context and leave both the decoding policy and the generated trace untouched: instruction rewriting, role conditioning, exemplar selection, output-format scaffolding, and retrieval injection all belong here. Retrieval is the outlier—it returns a different document set for different inputs, so the operator is stochastic even when the template is fixed. Its variance is attributed to the prompt unless the retrieved set is logged.

C. Decoding-space operators

Decoding-space operators leave the context untouched and reshape the per-step distribution before a token is drawn. Temperature scaling is a smooth reparametrisation, top-k and nucleus sampling restrict the support, and beam search replaces sampling with an approximate sequence search. These are the only operators applicable without changing a single character of the prompt, which makes them the natural control condition for the experiments in Section V.

D. Trace-space operators

Trace-space operators change what the model generates before committing to an answer. Zero-shot CoT [16] adds a trigger phrase, tree-of-thoughts maintains a frontier of partial traces expanded under a value estimate, self-consistency draws several chains and aggregates them, and iterative refinement [17] repeats generation–critique cycles. None is free: each buys accuracy with tokens, forward passes, or both, and Table I records the exchange rate.

E. Composition and interaction

The three operator classes do not compose independently. Self-consistency requires $T > 0$—m greedy samples are m copies of one chain—so its trace-space effect is only defined relative to a decoding-space operator that supplies diversity. Few-shot CoT couples Φ and Ω because the exemplars carry the traces, tree-of-thoughts couples trace construction with value-guided search, and ReAct adds a tool environment on top of a context rewrite; none is a single-factor intervention. The main factorial of Section V therefore supports claims about complete configurations; single-operator claims are made only for self-consistency, whose two components (m and T) are crossed by the design itself. This is why we test two hypotheses:

- H1 Holding the algorithmic structure of a prompt fixed, paraphrasing its wording produces a smaller change in task accuracy than moving between technique configurations (cells of Table I).
- H2 The interaction between the trace and decoding components of a configuration is significant, so reporting a trace-space technique without its decoding configuration underdetermines the result.

IV. Algorithmic Framework

A. Base distribution

Let $\mathcal{V}$ be the vocabulary and let $y = (y_1, \ldots, y_{|y|}) \in \mathcal{V}^*$ be an output sequence. An autoregressive LLM factorises

$$p_\theta(y \mid P, x) = \prod_{t=1}^{|y|} p_\theta(y_t \mid y_{<t}, P, x), \tag{1}$$

where each factor is obtained from a logit vector $z_t \in \mathbb{R}^{|\mathcal{V}|}$ by

$$p_\theta(y_t = v \mid y_{<t}, P, x) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}. \tag{2}$$

Equations (1) and (2) contain every quantity a prompting technique can touch: it either changes z_t through the conditioning arguments, changes the map from z_t to a sampled token, or changes which sequences are generated and how they are combined.

B. Prompt-space operators as logit displacement

Let $\Phi : P \mapsto P'$ and write $z_t(P)$ for the logits obtained under context P. The operator induces a displacement

$$\Delta_t(\Phi) = z_t(P') - z_t(P) \in \mathbb{R}^{|\mathcal{V}|}. \tag{3}$$

Because the softmax in (2) is invariant to adding a constant to every logit, only the component of Δ_t orthogonal to $\mathbf{1}$ has any behavioural consequence. The effective displacement is therefore the centred vector

$$\hat{\Delta}_t(\Phi) = \Delta_t(\Phi) - \frac{1}{|\mathcal{V}|}\big(\mathbf{1}^\top \Delta_t(\Phi)\big)\mathbf{1}, \tag{4}$$

and $\|\hat{\Delta}_t\|_2$ is a scalar summary of how hard the rewrite pushed on the model at step t. A prompt edit that raises every logit uniformly—one that only lengthens the context without adding discriminative content—has $\hat{\Delta}_t \approx 0$ and cannot change behaviour no matter how many tokens it costs.

A second, coarser summary is the divergence induced at the distribution level. Writing $p_t = p_\theta(\cdot \mid y_{<t}, P, x)$ and $p'_t = p_\theta(\cdot \mid y_{<t}, P', x)$,

$$D_\Phi = \mathbb{E}_x \left[|y|^{-1} \sum_{t=1}^{|y|} D_{\mathrm{KL}}\left(p'_t \,\|\, p_t\right) \right], \tag{5}$$

which we report per token so that prompts of different lengths remain comparable. Attention offers a third view: the mass a head places on a newly inserted span tells us whether the model actually reads the added text.

C. Decoding-space operators as measure transforms

Fix the logits and let Ψ act on them. Temperature scaling produces

$$q_T(v) = \frac{\exp(z_v/T)}{\sum_u \exp(z_u/T)}, \tag{6}$$

a Gibbs family whose entropy $H(q_T)$ is non-decreasing in T, approaching a point mass on $\arg\max_v z_v$ as $T \to 0^+$ and the uniform distribution as $T \to \infty$. Temperature is thus a continuous interpolation between the maximisation and uniform-exploration regimes.

Truncation operators behave differently. Let π order $\mathcal{V}$ by decreasing probability. Top-k retains $S_k = \{\pi_1, \ldots, \pi_k\}$ and nucleus sampling retains the smallest prefix whose mass reaches p,

$$S_p = \{\pi_1, \ldots, \pi_\kappa\}, \quad \kappa = \min\left\{ j : \sum_{i \le j} q(\pi_i) \ge p \right\}, \tag{7}$$

after which the retained mass is renormalised. Both discard the tail, but in different ways.

Proposition 1 (Support and entropy bounds). *Let $q^{(k)}$ be the renormalisation of q onto S_k. Then $|\operatorname{supp}(q^{(k)})| \le k$ and $H(q^{(k)}) \le \log k$, both independent of the context. For nucleus sampling the analogous bound is $H(q^{(p)}) \le \log \kappa(x,t)$, where κ is a function of the step-level distribution and is therefore not fixed in advance.*

Proposition 1 explains an asymmetry that is often observed but rarely stated: top-k imposes a hard, context-independent ceiling on per-step diversity, whereas nucleus sampling lets the ceiling float with the model's own confidence, contracting the support where the model is certain and widening it where it is not. Reporting a single top-p value without the resulting κ statistics therefore conveys much less information than it appears to.

D. Trace-space operators as marginalisation and search

Let r denote an intermediate trace. The answer distribution the model actually defines is the marginal

$$p_\theta(y \mid P, x) = \sum_r p_\theta(y \mid r, P, x)\, p_\theta(r \mid P, x). \tag{8}$$

Direct answering approximates $\arg\max_y$ of (8) without ever materialising r. Greedy CoT does something different: it materialises one trace and returns the answer attached to it, which approximates

$$\hat{y}_{\mathrm{CoT}} = \arg\max_y \max_r p_\theta(y, r \mid P, x), \tag{9}$$

the joint mode rather than the marginal mode. The two need not coincide.

Proposition 2 (Mode mismatch). *There exist distributions for which $\arg\max_y \max_r p(y,r) \ne \arg\max_y \sum_r p(y,r)$.*

Proof. Take two answers and three traces with $p(y_1, r_1) = 0.30$ and the remaining joint mass spread uniformly over the three traces for y_2. The joint mode selects y_1 (0.30), but the marginals satisfy $p(y_1) = 0.30 < p(y_2) = 0.70$. ■

Proposition 2 is the algorithmic content of self-consistency. Drawing m traces and taking the plurality answer is a Monte Carlo estimator of the marginal mode, so the technique corrects a systematic bias in greedy CoT rather than merely averaging away noise. Its reliability follows from a standard concentration argument.

Proposition 3 (Vote concentration). *Let $y^\star$ maximise the marginal with mass $q^\star$, let q' be the largest mass on any other answer, and let $\gamma = q^\star - q' > 0$. For m i.i.d. traces, the plurality vote returns $y^\star$ with probability at least $1 - (|\mathcal{Y}| - 1)\exp(-m\gamma^2/2)$.*

Proof. For each competing answer apply Hoeffding's inequality to the difference of the two empirical frequencies, which has mean γ and bounded increments, then take a union bound over the at most $|\mathcal{Y}| - 1$ competitors. ■

The useful range of m is governed by the margin γ rather than by task difficulty—items with a small margin need many samples—which makes adaptive sample allocation profitable. Self-consistency, though, cannot repair a distribution whose marginal mode is wrong: increasing m only sharpens convergence to y^*, so such errors have to be addressed in prompt space or trace space instead.

Tree-of-thoughts replaces sampling with search. With branching factor b, depth d, and a value estimate $v(\cdot)$ over partial traces, the method returns

$$\hat{y}_{\mathrm{ToT}} = \arg\max_y \max_{r \in \mathcal{R}_{b,d}} v(r)\, p_\theta(y \mid r, P, x), \tag{10}$$

where $\mathcal{R}_{b,d}$ is the set of traces the search actually visits. The model's own trace probability is partly replaced by an external value signal, so ToT inherits the quality of v. ReAct is the same substitution carried out by the environment instead of a value function: the trace interleaves actions with observations $o_{1:n}$, and the conditioning distribution becomes $p_\theta(y \mid P, x, o_{1:n})$ with o_i drawn from a tool. This is why ReAct reduces errors that arise from missing information and leaves errors of reasoning largely intact.

E. Cost model

Let C count forward passes and N count generated tokens. Direct answering has $C = 1$, $N = O(|y|)$; CoT has $C = 1$, $N = O(|r| + |y|)$; self-consistency multiplies both by m; ToT costs $C = O(bd)$ passes plus the value calls. Because $|r| \gg |y|$ on reasoning tasks, the dominant term is usually trace length rather than passes, so accuracy per thousand generated tokens is a more faithful efficiency measure than accuracy per call; both are reported in Section VI.

F. Separating algorithmic from lexical effects

Let Φ_{lex} range over paraphrases that preserve algorithmic structure—same exemplar count, same trace shape, same output format—and Φ_{alg} over changes of cell in Table I. Fitting a two-factor model to the accuracy A over these factors and their interaction, we define the algorithmic effect ratio

$$\mathrm{AER} = \frac{\sigma^2_{\mathrm{alg}}}{\sigma^2_{\mathrm{alg}} + \sigma^2_{\mathrm{lex}}}, \tag{11}$$

where each term is a variance component. An AER near 1 means the benefit survives rewording; a value near 0.5 means half of a reported gain is a property of the particular sentence. Because the implemented configurations differ in more than one operator wherever they differ at all, the AER characterises complete configurations and licenses no single-operator attribution. The distribution-level quantities D_Φ of (5) and the κ statistics of Proposition 1 remain part of the framework, but the empirical protocol of Section V-E reports them only where the model API supplies per-token probabilities; the benchmark of Section V instead reports parse rates, token cost, and latency alongside the AER.

V. Experimental Setup

A. Theoretical Framework and Justification

The taxonomy of Section III and the framework of Section IV make one testable claim: the operator configuration of a technique determines its behavioural consequences more than the surface wording does. Testing this claim requires a design that (i) implements configurations spanning all three operator classes (for composite techniques, with claims restricted to complete configurations, per Section V-C), (ii) includes a lexical factor that holds the algorithm fixed while rewording the prompt, so that the algorithmic effect ratio of Equation (11) can be estimated, and (iii) records accuracy, parse rates, token counts, and wall-clock latency, so that a prompt that moves behaviour without moving it usefully is distinguishable from an inert one. The factorial design below satisfies these requirements and tests hypotheses H1 and H2 of Section III.

B. Research Design

The design is factorial with three factors corresponding to the three axes of the taxonomy. The first factor is the technique configuration, with four levels implemented in the local benchmark:

1) Direct answering (baseline, all operators set to identity),
2) Zero-shot chain-of-thought (Ω only, trigger phrase "Let's think step by step"),
3) Few-shot CoT ($\Phi + \Omega$ with $n = 2$ exemplars carrying reasoning chains),
4) Self-consistency ($\Omega + \Psi$ with $m = 4$ samples over few-shot CoT at $T = 0.7$, aggregating answers by plurality vote).

Tree-of-thoughts and ReAct appear in the taxonomy (Table I) but are excluded from the local benchmark: their search and tool loops require orchestration the Ollama harness does not provide, and their per-item cost is prohibitive on the 8 GB of GPU VRAM available on the laptop used here (16 GB of system RAM). The four implemented configurations still span all three operator classes (Φ alone, Ω alone, $\Phi+\Omega$, and $\Omega+\Psi$), which is sufficient to test both hypotheses.

The second factor is lexical: four paraphrases of each GSM8K template and two paraphrases of each HumanEval template, checked to preserve exemplar count, trace shape, and output format, so the algorithmic content is identical by construction; this operationalises Φ_{lex} in the AER equation. The smaller paraphrase count on HumanEval bounds the compute budget on the local hardware while keeping at least two lexical levels, the minimum needed to estimate the lexical variance component.

The third factor is decoding: temperature $T \in \{0.0, 0.3, 0.7, 1.0\}$ on GSM8K and greedy decoding ($T = 0$) on HumanEval, with self-consistency evaluated only at its defined setting $T = 0.7$. Stochastic sampling uses a nucleus threshold $p = 0.9$ and top-$k = 40$. Greedy decoding is the reference cell. This factor operationalises Ψ and enables the interaction test required by H2, because self-consistency's effect is only defined relative to a non-greedy configuration that supplies diverse chains.

All model weights are frozen; no fine-tuning is performed at any stage. Within each cell of the design, the same evaluation set is used under a fixed random seed, and every response is logged with its generated token count and wall-clock latency.

C. Scope of causal claims

Every implemented configuration is a single operator or a fixed composition (few-shot CoT couples Φ and Ω; self-consistency couples Ω and Ψ), so the main factorial supports claims about complete configurations, not about individual operators in isolation. Self-consistency is the exception: its two components (m and T) are already crossed by the design. No single-operator attribution is claimed for any other technique.

D. Data Collection Procedure

We select two benchmark datasets that cover the task regimes in which prompting techniques are most frequently evaluated.

GSM8K [18] is a set of 8,792 training and 1,319 test grade-school arithmetic problems (MIT license); accuracy is exact match of the final numeric answer, extracted by a shared regular expression.

HumanEval [19] is a program-synthesis benchmark of 164 hand-written problems; accuracy is pass@1, computed by executing each generated program against the provided unit tests in an isolated subprocess with a 15 s timeout.

We draw a plain uniform-random sample of 100 items from the GSM8K test split under a single fixed seed (42) and evaluate all 164 HumanEval problems, which are few enough to be covered exhaustively; sampling is not stratified. The evaluation set is identical across every cell of the design and is fixed before analysis: the seed and the complete item-ID lists are recorded with the evaluation logs, so any run can reconstruct the same set. These sizes balance power against cost: at accuracies near 0.5, $n = 100$ yields 95% confidence intervals of roughly ± 10 points under the paired bootstrap of Section V-E, and $n = 164$ roughly ± 8 points.

TABLE I
TAXONOMY OF PROMPT ENGINEERING TECHNIQUES BY MODIFIED ALGORITHMIC OBJECT

Technique	**Operator**	**Object modified**	**Forward passes**	**Added tokens**	**Stochastic?**
Instruction rewriting	Φ	conditioning context	1	$O(\|I\|)$	no
Role / persona conditioning	Φ	conditioning context	1	$O(1)$	no
Few-shot exemplars	Φ	conditioning context	1	$O(n\bar{\ell})$	no
Format scaffolding	Φ	conditioning context	1	$O(1)$	no
Retrieval augmentation	Φ	conditioning context	1+retrieval	$O(n_d\bar{\ell}_d)$	yes
Temperature scaling	Ψ	per-step distribution	1	0	yes
Top-k truncation	Ψ	per-step support	1	0	yes
Nucleus (top-p) sampling	Ψ	per-step support	1	0	yes
Beam search	Ψ	sequence-level objective	b	0	no
Constrained decoding	Ψ	per-step support	1	0	either
Zero-shot CoT	Ω	reasoning trace	1–2	$O(1)$	no
Few-shot CoT	$\Phi + \Omega$	context and trace	1	$O(n\bar{\ell}_r)$	no
Least-to-most prompting	Ω	trace decomposition	s	$O(s)$	no
Self-consistency	$\Omega + \Psi$	trace ensemble	m	0	yes
Tree-of-thoughts	$\Omega + \Psi$	trace search	$O(bd)$	$O(bd\bar{\ell}_r)$	either
ReAct	$\Phi + \Omega$	trace and context	$O(n_a)$	$O(n_a\bar{\ell}_o)$	yes
Self-refine	$\Phi + \Omega$	trace and context	$2c$	$O(c\bar{\ell}_r)$	no

Models. Three small open-weight models are evaluated, spanning a $3\times$ parameter range: Llama 3.2 1B (1.2 billion parameters) and Llama 3.2 3B (3.2 billion parameters), two general-purpose instruction-tuned models from Meta, and Qwen2.5-Coder-3B (3.09 billion parameters), a domain-specialised code and mathematics instruction model from Alibaba. A reasoning-distilled third model (DeepSeek-R1 1.5B) was originally planned, but under the structured evaluation templates of Section V-B it produced unbounded "thinking" traces—internal rationale continued well beyond the answer slot, inflating generation cost and destabilising answer extraction. We adopted a specialised code/math instruction baseline instead, which keeps the third scale at the same parameter order while testing how a domain-specialised prior interacts with the operator classes of Section III. Because Qwen2.5-Coder-3B substitutes a domain-specialised model for the planned third scale rung, the direct scale-versus-technique comparison of Section VI-A rests on the two Llama models (1B and 3B); the coder model contributes the specialisation axis rather than a third point on the scale ladder. The models are served locally as GGUF quantisations (Q4_K_M or Q8_0 precision) through Ollama, which uses the llama.cpp inference engine with CUDA acceleration on the NVIDIA GPU. Small open-weight models are required because the full benchmark—three models, two tasks, and roughly 18,000 generations—must run on a single consumer machine. The Ollama harness does not expose per-token log-probabilities, so the distributional quantities D_Φ and κ defined in Section IV are outside the measured scope of this study; the empirical protocol therefore reports accuracy, parse rates, token cost, and latency.

E. Data Processing and Analysis Tools

Evaluation harness. Inference runs through a purpose-built Python 3 harness (`run_experiments.py`) that calls the local Ollama HTTP API (`/api/generate`) with streaming disabled and the decoding options fixed per cell. GSM8K answers are extracted with a shared regular expression (preferring the value after the '####' delimiter); parse failures are counted as errors and reported separately. HumanEval completions are executed against the provided unit tests in an isolated subprocess with a 15 s timeout, mirroring the pass@1 protocol of the original benchmark. Self-consistency is aggregated differently per task: on GSM8K, a plurality vote over the normalised extracted numeric answers, with ties between equally frequent answers resolved in first-seen (insertion) order; on HumanEval, where generated programs cannot be reliably canonicalised for voting, a strict-majority rule in which an item counts as solved only when at least three of its four programs pass the unit tests.

Statistical analysis. Confidence intervals come from a paired bootstrap over items with 10,000 resamples. Pairwise comparisons across the algorithmic factor use Holm–Bonferroni correction. Variance components for the AER are estimated by the method of moments from the two-way (technique $\times$ paraphrase) analysis of variance at the reference decoding cell.

Hardware. All experiments run locally on a single consumer gaming laptop (Acer Predator) with an NVIDIA GeForce RTX 4060 GPU (8 GB of VRAM) and 16 GB of system RAM, running Linux, using CUDA acceleration through Ollama (llama.cpp backend); no GPU cluster or cloud inference is used. Wall-clock latency is recorded per item as a secondary cost metric alongside generated token counts.

Reproducibility. All prompt templates, model responses, evaluation logs, and analysis scripts are available from the authors on request; the harness is resumable, every record carries its cell and item identifiers, and the evaluation seed is fixed in the harness configuration.

VI. RESULTS AND ANALYSIS

Table II reports accuracy for the four configurations at the reference decoding cell ($T = 0$, $p = 1.0$), except self-

consistency, which is reported at its defined setting $T = 0.7$ because greedy decoding is undefined for $m > 1$ (Section III). The benchmark produced roughly 18,000 generations across three models and two tasks. Answer extraction with the regular expressions of Section V-E succeeded on 96–100% of responses across the cells of the factorial (at least 99.5% at the reference cell); parse failures are counted as errors. Headline cells vary across the four GSM8K paraphrases—1B direct 12–42%, 1B zero-shot CoT 45–50%, 1B few-shot CoT 8–25%, 1B self-consistency 4–29%, 3B direct 39–77%, 3B zero-shot CoT 75–82%, 3B few-shot CoT 66–79%, 3B self-consistency 78–86%, coder direct 3–11%, coder zero-shot CoT 72–81%, coder few-shot CoT 68–75%, coder self-consistency 74–81%—while HumanEval cells move by at most 4.2 points across its two paraphrases. Two findings organise the results. First, the algorithmic effect of a technique is comparable in size to the effect of model scale at this size class: on GSM8K, zero-shot CoT raises the 1B model from 12% to 50%, a gain of the same order as the 41-point improvement from scaling the model from 1B to 3B (Section VI-A). Second, the effect is task-dependent: trace- and ensemble-based techniques deliver their largest gains on GSM8K and are roughly neutral, or negative, on HumanEval.

A. Main comparison

On GSM8K the technique ordering depends on scale. Zero-shot CoT is the strongest configuration at the smallest scale: it raises the 1B model from 12% to 50%, a gain of 38 points that nearly matches the 41-point improvement from scaling the model from 1B to 3B under direct answering (12% to 53%). Few-shot CoT helps the 3B model (53% to 70%) and the coder model (6% to 71%); the apparent drop at the 1B model (12% to 8%) lies inside the bootstrap interval and is not distinguishable from noise at this sample size. Self-consistency adds a further gain over few-shot CoT at the 3B model (70% to 79%), whose uncorrected 95% paired-difference interval excludes zero, and a smaller gain at the coder scale (71% to 74%), whose interval includes zero, at a fourfold token cost (Tables II and IV). The coder model's direct-answering accuracy on GSM8K is unusually low (6%) and rises steeply with trace construction, evidence that a specialised prior does not by itself make arithmetic tractable without intermediate reasoning.

On HumanEval the ordering is compressed. No technique improves pass@1 by more than three points, zero-shot CoT is neutral or negative, and self-consistency is the weakest configuration at every scale (Table II). The shortfall relative to direct answering is significant in uncorrected comparisons at the 1B model (-11 points; 95% CI $[-16.5, -6.1]$) and the 3B model (-8.5; $[-14.0, -3.0]$) and directionally consistent at the coder model; its strength comes from both the individual paired differences and their cross-model consistency. Paired-bootstrap tests of each technique against direct answering at the reference cell, with Holm–Bonferroni correction within each model–task block, confirm the headline gains: zero-shot CoT is significant at all three GSM8K scales (corrected $p < 0.001$) and at the 1B HumanEval scale (corrected $p = 0.027$), and few-shot CoT is significant at the 3B (corrected $p = 0.008$) and coder (corrected $p < 0.001$) GSM8K scales; no other technique effect survives correction. Self-consistency was excluded from this corrected family because its reference cell ($T = 0.7$) differs from the $T = 0$ cell at which zero-shot and few-shot CoT are compared with direct answering; the confidence intervals reported for self-consistency in this section are uncorrected. Differences from direct answering are evaluated against the bootstrap intervals in the table note, which are wide at $n = 100$; only differences exceeding the interval width are treated as evidence of an effect.

B. Decoding interactions

Table III and Fig. 1a show the interaction between temperature and sampling strategy on GSM8K under nucleus sampling ($p = 0.9$). Single-sample few-shot CoT accuracy changes little across temperature at all three scales: the endpoint movements (1B 8% to 6%, coder 71% to 62%, 3B 70% to 66%) and the apparent peak at $T = 0.3$ at the 3B scale all lie inside the bootstrap interval and are not distinguishable from noise. Self-consistency requires $T > 0$ and is evaluated at its defined setting $T = 0.7$. The ordering of the two curves—self-consistency lies above single-sample few-shot CoT at the 3B (79% vs. 72%) and coder (74% vs. 69%) scales but below it at the 1B scale (4% vs. 6%)—is consistent in direction only at the two larger scales; paired-bootstrap 95% intervals on each difference include zero (3B $[-2, +16]$, coder $[-4, +14]$, 1B $[-7, +3]$), so H2 is consistent with the direction of the ordering at the 3B and coder scales but is not statistically confirmed at this sample size.

C. Cost

Table IV converts accuracy into generation cost on GSM8K at the reference decoding cell. Reasoning traces are the dominant cost: few-shot CoT emits several times as many tokens as direct answering, and self-consistency multiplies that by $m = 4$. Wall-clock latency tracks tokens closely because all three models run on the same CUDA backend. Accuracy per thousand generated tokens is therefore highest for direct answering and lowest for self-consistency, and the ranking by cost-adjusted accuracy can differ from the ranking by raw accuracy in Table II.

D. Algorithmic effect ratio

Table V reports the algorithmic effect ratio (Equation (11)) and its variance components at the reference decoding cell, estimated by the method of moments from the technique $\times$ paraphrase rectangle. An AER near one means the technique gains survive rewording; a value near 0.5 means roughly half of a reported gain is a property of the particular sentence. On GSM8K the ratio is near one for all three models (0.82–1.00). On HumanEval the estimate is degenerate for the 3B model, where both variance components vanish because the technique configurations are nearly flat, and low for the coder model (0.25), where lexical variance exceeds algorithmic variance;

TABLE II

ACCURACY BY TECHNIQUE, TASK, AND MODEL AT THE REFERENCE DECODING CELL (GREEDY, $T = 0$, $p = 1.0$); SELF-CONSISTENCY IS REPORTED AT ITS DEFINED SETTING $T = 0.7$, $m = 4$, AT WHICH GREEDY DECODING IS UNDEFINED FOR $m > 1$ (SECTION III). VALUES ARE PERCENTAGES: GSM8K IS EXACT-MATCH ACCURACY ON THE 100-ITEM SAMPLE, HUMANEVAL IS PASS@1 ON ALL 164 PROBLEMS. ALL VALUES ARE PARAPHRASE 1 (OF FOUR GSM8K PARAPHRASES, TWO FOR HUMANEVAL); THE PER-PARAPHRASE RANGES ARE GIVEN IN SECTION VI. THE MODEL-SCALE COMPARISON APPEARS IN FIG. 1B.

Technique	**Llama 3.2 1B**		**Llama 3.2 3B**		**Qwen2.5-Coder-3B**	
	GSM8K	HumanEval	GSM8K	HumanEval	GSM8K	HumanEval
Direct answering	12.0	31.1	53.0	50.0	6.0	80.5
Zero-shot CoT	50.0	22.0	75.0	48.8	79.0	79.9
Few-shot CoT	8.0	31.1	70.0	50.0	71.0	82.9
Self-consistency	4.0	20.1	79.0	41.5	74.0	75.0

[a]95% paired-bootstrap CIs over items: ± 10 points (GSM8K, $n = 100$) and ± 8 points (HumanEval, $n = 164$) at accuracies near 0.5.

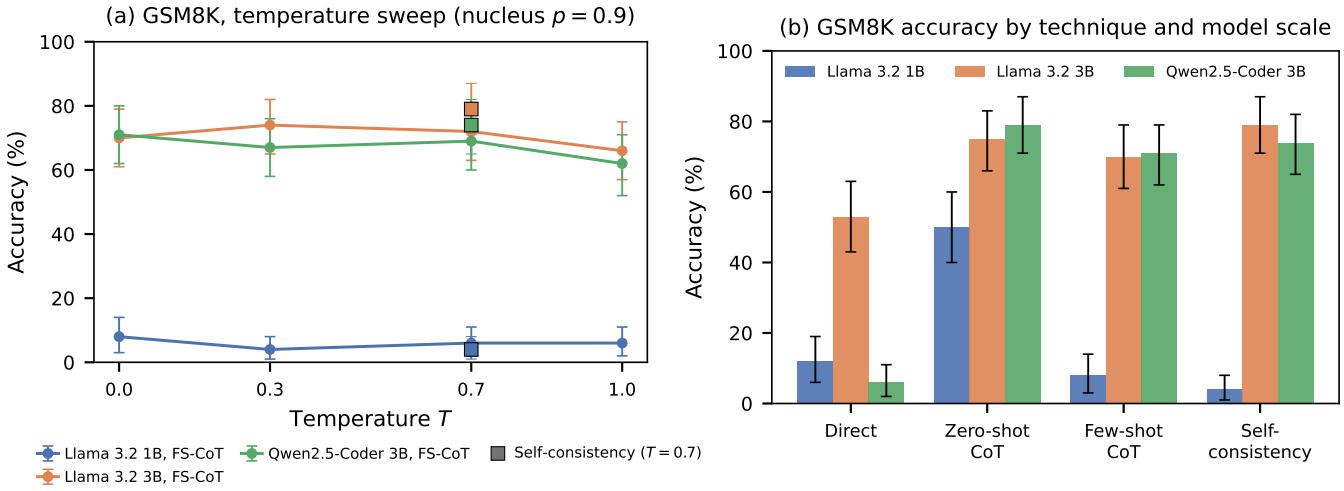

Fig. 1. (a) GSM8K accuracy versus temperature for single-sample few-shot CoT and self-consistency decoding under nucleus sampling ($p = 0.9$); error bars are 95% bootstrap confidence intervals over items, and square markers at $T = 0.7$ denote self-consistency, which requires $T > 0$ and is evaluated at its defined setting $T = 0.7$; single-sample accuracy moves little with temperature at all three scales, with the endpoint movements within the bootstrap interval. (b) GSM8K accuracy by technique and model scale at the reference decoding cell; the zero-shot-CoT gain at the 1B scale (12% to 50%) is of the same order as the gain from scaling the model from 1B to 3B under direct answering (12% to 53%).

TABLE III

GSM8K ACCURACY (%) UNDER THE TEMPERATURE SWEEP (PARAPHRASE 1, NUCLEUS $p = 0.9$). FEW-SHOT COT IS SINGLE-SAMPLE; SELF-CONSISTENCY ($m = 4$) IS EVALUATED ONLY AT ITS DEFINED SETTING $T = 0.7$.

	Llama 3.2 1B		**Llama 3.2 3B**		**Qwen2.5-Coder-3B**	
T	FS-CoT	SC	FS-CoT	SC	FS-CoT	SC
0	8.0	—	70.0	—	71.0	—
0.3	4.0	—	74.0	—	67.0	—
0.7	6.0	4.0	72.0	79.0	69.0	74.0
1	6.0	—	66.0	—	62.0	—

TABLE IV

GENERATION COST ON GSM8K AT THE REFERENCE DECODING CELL: MEAN GENERATED TOKENS PER ITEM AND MEAN WALL-CLOCK LATENCY IN SECONDS PER ITEM.

	1B		**3B**		**Qwen2.5-Coder-3B**	
Technique	Tokens	Sec.	Tokens	Sec.	Tokens	Sec.
Direct answering	34	0.4	79	1.0	33	0.5
Zero-shot CoT	178	1.5	185	2.1	244	3.3
Few-shot CoT	168	1.4	173	2.1	126	1.7
Self-consistency	730	5.5	722	7.9	503	6.1

only the 1B model shows a high ratio (0.98) on the code task. The lexical factor is evaluated with four paraphrases on GSM8K and two on HumanEval, so the HumanEval estimates are the noisier of the two.

Because each variance component is estimated from cell means over only 100 items (GSM8K) or 164 items (HumanEval) and as few as two paraphrase levels, an item-level bootstrap (5,000 resamples) yields wide 95% intervals (1B GSM8K [0.90, 1.00]; 1B HumanEval [0.23, 1.00]; 3B GSM8K [0.40, 1.00]; Qwen2.5-Coder-3B GSM8K [1.00, 1.00]; Qwen2.5-Coder-3B HumanEval [0.00, 1.00]); the point

estimates should be read as indicative rather than precise. The 3B HumanEval value is undefined rather than a real 0.50: in 42% of resamples both variance components collapse to zero, so the reported ratio is a reporting convention for an unidentifiable quantity, not a measurement.

TABLE V
ALGORITHMIC EFFECT RATIO (EQUATION (11)) AND ITS VARIANCE COMPONENTS AT THE REFERENCE DECODING CELL. SELF-CONSISTENCY IS EXCLUDED BECAUSE ITS REFERENCE SETTING DIFFERS ($T = 0.7$).

Model	Task	AER	σ^2_{alg}	σ^2_{lex}
1B	GSM8K	0.99	0.0281	0.0003
1B	HumanEval	0.98	0.0019	0.0000
3B	GSM8K	0.82	0.0060	0.0013
3B	HumanEval	0.50	0.0000	0.0000
Qwen2.5-Coder-3B	GSM8K	1.00	0.1583	0.0000
Qwen2.5-Coder-3B	HumanEval	0.25	0.0000	0.0001

[a]For the 3B HumanEval row both variance components vanish (no technique effect is detectable), so the ratio is undefined and is reported as 0.50 by convention. [b]For the Qwen2.5-Coder-3B GSM8K row the lexical variance component is identically zero across all 5,000 resamples, pinning the ratio at its upper bound of 1.00—the same variance-clipping degeneracy as the 3B HumanEval row, in the opposite direction.

VII. DISCUSSION

The results support the paper's central claim: the algorithmic structure of a technique, not its wording, drives the measured effect, and at this size class that effect is comparable in magnitude to model scale. Because the evaluated set pairs two general-purpose Llama models with a domain-specialised coder model (Qwen2.5-Coder-3B), the comparison also tests whether trace-space techniques behave differently under a specialised prior. All three models run under identical CUDA-accelerated decoding on a single machine, so cost and latency are directly comparable.

Trace construction is where the cost–quality frontier is steepest. On GSM8K, few-shot CoT emits roughly two to five times as many tokens as direct answering (Table IV), and self-consistency multiplies that by $m = 4$ while adding only a fraction of the accuracy it costs in tokens. Per token, direct answering is the most efficient configuration; few-shot CoT buys a large accuracy gain for a moderate token premium, and self-consistency buys a small gain at a fourfold cost. The ranking by cost-adjusted accuracy therefore depends on the budget, which is why token cost and latency are reported alongside accuracy throughout.

There is a finding in the flat HumanEval column too. Techniques that restructure the trace help where the answer is the product of intermediate computation (GSM8K), and help little, or hurt, where the output is checkable by execution (code). The individual HumanEval deltas lie inside the reported interval, so the strength of this reading lies in the direction being consistent across all three models rather than in any single comparison. Robustness to rewording is high when the AER is near one—the algorithmic structure, not the sentence, carries the effect—and the temperature sweep shows why the decoding configuration must be reported with any technique: single-sample accuracy moves little with temperature at the scales tested (all endpoint changes lie inside the reported interval), while sampling-based techniques such as self-consistency require a nonzero temperature to be defined at all.

Self-consistency's ensemble framing carries a bias dimension of its own. It cannot repair a systematically wrong marginal mode (Proposition 2): majority voting sharpens convergence to whichever answer the ensemble favours, so a prompt that elicits a consistent but incorrect bias gets amplified rather than corrected. Parse failures, counted as errors and reported separately, are a second source of systematic loss—a technique that raises accuracy only by changing the answer format trades one kind of failure for another unless parse rates are reported.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The main threat is external validity: all results come from a narrow set of small open-weight models (Llama 3.2 1B and 3B, and the code-specialised Qwen2.5-Coder-3B), so the interaction structure—particularly the temperature effects—may differ for larger models or for API-hosted models with hidden sampling defaults.

Power is limited. At $n = 100$ the GSM8K intervals are roughly ± 10 points and at $n = 164$ the HumanEval intervals are roughly ± 8 points, so small but real effects will not be detected. The paraphrase factor is small (four paraphrases on GSM8K, two on HumanEval), and the two HumanEval paraphrases give coarse lexical variance estimates. On HumanEval the algorithmic effect is small enough that the AER is degenerate for the 3B model (both variance components vanish) and is weakly algorithmic for the coder model (0.25).

The Ollama harness does not expose per-token log-probabilities, so the distributional quantities D_Φ and κ defined in Section IV are not measured here; the empirical protocol is restricted to accuracy, parse rates, tokens, and latency. Tree-of-thoughts and ReAct are absent from the benchmark, so the search- and tool-based operator classes are covered only analytically. A single regular-expression parser counts its failures as errors, HumanEval execution is sandboxed with a 15 s timeout that may reject correct but slow solutions, and the 8 GB of GPU VRAM limits the model scale that can be evaluated. The evaluation set is fixed before analysis and released with the artefacts, and costs are single-run estimates on a single machine.

IX. CONCLUSION

Framed algorithmically, prompt engineering techniques act as operators on the conditional output distribution rather than as incidental changes to a string of text. Our controlled factorial study shows their measured effects are separable from both prompt wording and decoding configuration: the algorithmic effect of a technique is comparable in size to the effect of model scale in the 1B–3B range; it concentrates on reasoning tasks and is neutral or negative where the output is checkable by execution; and temperature interacts with sampling-based techniques, with self-consistency defined only

at a nonzero temperature. The benchmark—four techniques, two tasks, three locally served small models (two general-purpose Llama scales and a domain-specialised coder), and roughly 18,000 generations—runs end to end on a single consumer gaming laptop with an 8 GB VRAM GPU, which puts the protocol in reach of researchers without GPU clusters. We recommend future studies report the algorithmic effect ratio, parse rates, token cost, wall-clock latency, and the full decoding configuration—the same protocol used here—so that gains become attributable, comparable, and reproducible at whatever scale is available.

ACKNOWLEDGMENT

The authors thank the developers of the open-weight models and public benchmarks used in this study, and the maintainers of the evaluation harness on which the experiments were run.

REFERENCES

[1] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[2] S. Min et al., "Rethinking the role of demonstrations: What makes in-context learning work?," in *Proc. Conf. Empir. Methods Nat. Lang. Process.* (EMNLP), 2022, pp. 10748–10760.

[3] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 35, 2022, pp. 24824–24837.

[4] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.

[5] S. Yao et al., "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 36, 2023, pp. 11809–11822.

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.

[7] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[8] Y. Zhou et al., "Large language models are human-level prompt engineers," in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.

[9] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2020.

[10] P. Sahoo et al., "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024, arXiv:2402.07927. [Online]. Available: https://arxiv.org/abs/2402.07927

[11] Y.-Y. Liu et al., "A comprehensive taxonomy of prompt engineering techniques for large language models," *Front. Comput. Sci.*, vol. 20, no. 3, Art. no. 2003601, 2026, doi:10.1007/s11704-025-50058-z.

[12] L. Mei et al., "A survey of context engineering for large language models," 2025, arXiv:2507.13334. [Online]. Available: https://arxiv.org/abs/2507.13334

[13] W. Du, Y. Yang, and S. Welleck, "Optimizing temperature for language models with multi-sample inference," in *Proc. Int. Conf. Mach. Learn.* (ICML), 2025.

[14] P. Liu et al., "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.

[15] S. Schulhoff et al., "The prompt report: A systematic survey of prompting techniques," 2024, arXiv:2406.06608. [Online]. Available: https://arxiv.org/abs/2406.06608

[16] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 35, 2022, pp. 22199–22213.

[17] A. Madaan et al., "Self-refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 36, 2023, pp. 46534–46594.

[18] K. Cobbe et al., "Training verifiers to solve math word problems," 2021, arXiv:2110.14168. [Online]. Available: https://arxiv.org/abs/2110.14168

[19] M. Chen et al., "Evaluating large language models trained on code," 2021, arXiv:2107.03374. [Online]. Available: https://arxiv.org/abs/2107.03374